SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

DAY 9

Requirements Engineering Process

Student Book • Module II: Requirements Engineering

Learning focus
Understand how a software idea is transformed into clear, agreed, testable requirements before detailed design and development begin.

Running case study: College Event Management System (CEMS)

Designed for classroom projection, discussion, practical application, and revision.

1. Learning Outcomes

By the end of this session, you should be able to:

• Define requirements engineering and explain why it matters to software success.

• Describe the main stages of the requirements engineering process and the outputs of each stage.

• Identify stakeholders and prepare useful questions for requirements elicitation.

• Distinguish requirements discovery, analysis, specification, validation, and management.

• Apply the process to a realistic College Event Management System.

Key idea

Requirements engineering is the disciplined process of discovering, analysing, documenting, validating, and managing what a software system must do and the constraints under which it must operate. It connects a business problem to a shared, testable understanding of the solution.

2. What Is a Requirement?

A **requirement** is a statement of a capability, service, behaviour, or constraint that a system must satisfy. Requirements explain what stakeholders need, not necessarily the technical design used to meet that need.

Example: College Event Management System

Statement	Type / interpretation
Students can register for an event using their college email address.	Functional requirement: describes a system behaviour.
The event list should load within two seconds under normal load.	Non-functional requirement: states a performance expectation.
Only authorised event coordinators may publish an event.	Security/access-control requirement.
The system must comply with the college's data-retention policy.	Constraint / policy requirement.

Weak requirement: “The system should be user-friendly.” This is vague and difficult to test.
Improved requirement: “A first-time student should be able to complete event registration in no more than four clearly labelled steps.” The improved version is more observable and testable.

3. Why Requirements Engineering Matters

A project can use modern technology and talented developers yet still fail if the team builds the wrong thing. Requirements engineering reduces uncertainty before expensive design, coding, and testing decisions are made.

• **Shared understanding:** students, faculty, administrators, and developers agree on the problem.

• **Reduced rework:** misunderstandings are discovered earlier, when they are cheaper to correct.

• **Better planning:** scope, time, cost, skills, and dependencies become easier to estimate.

• **Testability:** each important requirement can be checked against expected behaviour.

• **Scope control:** new requests can be assessed instead of silently added.

• **Traceability:** the team can connect a stakeholder need to design, code, and test evidence.

Real-life analogy: Before constructing a house, the owner and architect clarify the number of rooms, budget, accessibility needs, materials, and legal constraints. Starting construction from a vague statement such as “build

a nice house" invites expensive changes. Software projects face the same problem.

4. The Requirements Engineering Process

The process is iterative. Teams may revisit earlier stages when new information appears. The following sequence is a useful baseline:

Stage	Main question	Typical output
1. Inception / feasibility	Why is the system needed, and is it viable?	Problem statement, goals, feasibility findings, initial scope
2. Elicitation	What do stakeholders need?	Interview notes, observations, user needs, initial requirements
3. Analysis and negotiation	Are the requirements clear, consistent, prioritised, and achievable?	Analysed requirements, resolved conflicts, priorities, models
4. Specification	How can requirements be documented precisely?	Software Requirements Specification (SRS), use cases, requirement IDs
5. Validation	Are these the right requirements, and can they be tested?	Review findings, approved requirements, corrections
6. Requirements management	How are changes and relationships controlled over time?	Version history, change log, traceability matrix, status

These stages are connected, not a one-way conveyor belt. For example, validation may reveal a missing stakeholder, sending the team back to elicitation. A change request may require analysis, revised specification, and re-validation.

Process flow

Business need → feasibility and scope → stakeholder discovery → elicitation → analysis and prioritisation → SRS / models → validation and approval → ongoing change management

5. Stage 1 — Inception and Feasibility

Inception establishes the problem, business objective, stakeholders, boundaries, assumptions, and initial success measures. Feasibility asks whether the proposed solution is realistic enough to pursue.

Questions to ask

• What problem are we solving? Who experiences it?

• What happens today, and what is painful or slow about it?

• What is included in the first release—and what is out of scope?

• Do we have the technical skills, budget, time, data, and permissions?

• How will success be measured?

Common feasibility dimensions

Dimension	Example question for CEMS
Technical	Can the team support login, event registration, and notifications with available technology?
Economic	Do expected benefits justify development and operating costs?
Operational	Will students and faculty be able and willing to use the system?
Schedule	Can the essential functions be delivered before the college fest?

Dimension	Example question for CEMS
Legal / policy	Can personal data be collected and retained according to applicable policy and law?

Example initial scope: students browse events and register; coordinators create events and view registrations; administrators manage roles. Online payment and external ticketing integrations are excluded from the first release unless separately approved.

6. Stage 2 — Requirements Elicitation

Elicitation means discovering needs from people, existing documents, current processes, and the environment in which the system will be used. Stakeholders may not know exactly what they need at the beginning, so the analyst must listen, probe, observe, and confirm.

Who are the stakeholders?

Stakeholder	Likely needs / concerns
Students	Find events quickly, register easily, receive confirmations, cancel when allowed.
Faculty coordinators	Create events, set capacity, check registrations, communicate updates.
College administration	Approve events, manage access, view reports, enforce policy.
IT support	Manage accounts, availability, backups, and support requests.
Security / data owner	Limit access to personal data and retain only approved information.

Elicitation techniques

Technique	How it works	Useful when
Interview	Ask an individual or small group structured and open questions.	Details, motivations, exceptions, or sensitive concerns matter.
Questionnaire / survey	Collect the same questions from many people.	You need broad feedback from a large student population.
Observation	Watch how the current process is actually performed.	Written procedures differ from real practice.
Document analysis	Review forms, policies, spreadsheets, reports, and existing systems.	Rules or current workflows are documented somewhere.
Workshop	Bring stakeholders together to map needs and resolve disagreements.	Different groups need to reach a shared decision.
Prototype	Show a rough screen or flow and ask users to react.	Users find it hard to describe an interface abstractly.

Good elicitation questions

• “Walk me through how you register for an event today.”

• “What happens if an event reaches its capacity?”

• “Can a student cancel a registration? Until when?”

• “Who is allowed to publish or edit an event?”

• “What information must appear in the confirmation?”

• “What should happen if the network fails during registration?”

Avoid leading questions. Instead of asking “You want a mobile app, right?”, ask “Which devices do students use, and what would make access convenient?” This discovers the need before assuming a solution.

7. Stage 3 — Analysis and Negotiation

Raw stakeholder statements are often incomplete, contradictory, duplicated, or too broad. Analysis turns them into requirements that are understandable, feasible, prioritised, consistent, and testable. Negotiation resolves conflicts between stakeholders and constraints.

Typical analysis activities

• Classify requirements as functional, non-functional, interface, data, or constraint requirements.

• Remove ambiguity and identify missing details, assumptions, dependencies, and exceptions.

• Check feasibility, consistency, completeness, and testability.

• Model processes or interactions using use cases, activity diagrams, or other suitable models.

• Prioritise requirements and negotiate trade-offs.

Example of a conflict

Students want to register for any event at any time. Coordinators want registrations to close once an event reaches capacity. The analyst clarifies the business rule and documents an agreed requirement: “The system shall prevent new registrations when confirmed registrations equal the event capacity and shall display a clear full-capacity message.”

Prioritisation: MoSCoW

Priority	Meaning	CEMS example
Must have	Essential for the release to work.	Students can view an event and register; capacity is enforced.
Should have	Important, but a workable temporary alternative exists.	Students receive an email confirmation.
Could have	Useful enhancement if time and budget permit.	Personalised event recommendations.
Won't have now	Explicitly excluded from this release.	Integrated online payments, if not needed for campus events.

Prioritisation is not simply a popularity contest. Consider business value, risk, dependencies, effort, legal obligations, and the minimum viable release.

8. Stage 4 — Requirements Specification

Specification records the agreed requirements in a form that stakeholders, developers, testers, and project managers can use. A common deliverable is the Software Requirements Specification (SRS). The level of detail depends on project risk, size, regulation, and delivery approach.

Example requirement statements

ID	Requirement	Acceptance / verification idea
FR-01	The system shall allow an authenticated student to view published events.	Given an authenticated student, when the event list is opened, published events are displayed.
FR-02	The system shall allow a student to register for an event with available capacity.	A valid registration is saved and the available seat count is updated.

ID	Requirement	Acceptance / verification idea
FR-03	The system shall prevent registration when capacity has been reached.	Attempting to register for a full event is rejected with a clear message.
NFR-01	The event list shall load within two seconds for 95% of requests under the agreed normal-load test.	Performance test records response times under a defined test setup.
SEC-01	Only users with the coordinator role shall create or publish events.	A student account attempting the action is denied.

What an SRS commonly contains

• Purpose, scope, definitions, references, and document overview.

• Product context, user classes, assumptions, dependencies, and constraints.

• Functional requirements with unique IDs and clear behaviour.

• Non-functional requirements such as performance, security, usability, reliability, and accessibility.

• External interfaces, data requirements, business rules, and error handling.

• Models or use cases, acceptance criteria, and traceability information.

A good requirement is clear, necessary, feasible, consistent, verifiable, traceable, and appropriately prioritised. Avoid words such as “fast,” “simple,” “secure,” and “user-friendly” unless they are defined with measurable criteria.

9. Stage 5 — Requirements Validation

Validation checks whether the documented requirements represent the stakeholders’ real needs and provide a suitable basis for building and testing the system. Validation is not the same as merely proofreading the SRS.

Validation techniques

Technique	Purpose
Requirements review / walkthrough	Stakeholders read and discuss requirements to find omissions, ambiguity, conflicts, or incorrect assumptions.
Prototyping	Users interact with a sample screen or flow to confirm expected behaviour.
Test-case derivation	Check whether a requirement can be turned into observable pass/fail tests.
Model validation	Check whether use cases and process models match real workflows.
Acceptance criteria review	Agree on evidence that will demonstrate the requirement is satisfied.

Validation checklist

• Is each requirement necessary and linked to a stakeholder or business goal?

• Is it clear enough that different readers interpret it the same way?

• Is it feasible within the agreed technology, budget, and schedule?

• Does it conflict with another requirement or policy?

• Can a tester verify it using observable evidence?

• Are normal, alternate, and error paths considered?

• Have the relevant stakeholders reviewed and approved it?

Verification vs validation: verification asks whether the requirements and work products are being specified correctly; validation asks whether they describe the right system for stakeholder needs. The two are related but not

interchangeable.

10. Stage 6 — Requirements Management

Requirements change because stakeholders learn more, business priorities shift, regulations evolve, or technical constraints emerge. Requirements management ensures that changes are assessed, approved, recorded, communicated, and reflected in affected project artefacts.

A simple change-control workflow

> Change request raised → impact analysis → priority and cost/schedule assessment → authorised decision → update requirement and version → update linked design, code, tests, and plans → communicate and verify

Example change request

Request: “Allow students to cancel event registrations.” Before accepting it, the team asks: Is cancellation allowed until the event starts or until a deadline? Does capacity reopen automatically? Is the coordinator notified? Do reports need to show cancellations? Which test cases and screens change?

Traceability

A Requirements Traceability Matrix (RTM) connects requirements to their origin and downstream artefacts. It helps answer: Where did this requirement come from? Has it been implemented? How will it be tested? What else will a change affect?

Requirement ID	Source	Design / component	Test case	Status
FR-02	Student interview	Registration service	TC-REG-02	Approved
FR-03	Coordinator workshop	Capacity validation	TC-CAP-01	Approved
NFR-01	College IT team	Event list API / UI	TC-PERF-01	Under review

Traceability is especially valuable when a requirement changes or when a team must demonstrate that all mandatory requirements have been addressed.

11. Case Study — CEMS From Idea to Requirements

Problem: the college currently announces events through multiple messaging groups and collects registrations using separate forms and spreadsheets. Students miss updates, coordinators struggle to track capacity, and duplicate entries occur.

Step A: define the problem and success

• Goal: provide one reliable place to discover and register for college events.

• Possible success measures: fewer duplicate registrations, faster registration tracking, and timely event updates.

• Initial scope: event catalogue, registration, capacity handling, coordinator event management, and basic reports.

Step B: discover stakeholder needs

Interview a student, a faculty coordinator, and an administrator. Observe the current registration workflow. Review existing forms, event approval rules, and spreadsheets. Record the source of each need.

Step C: analyse and prioritise

Group requirements by function and quality attribute. Clarify registration limits, cancellation rules, user roles, event publication, and what happens when capacity is reached. Mark release-critical requirements as Must have.

Step D: specify

Create an SRS with unique requirement IDs, clear wording, assumptions, constraints, use cases, and acceptance criteria. Keep design decisions separate unless they are required constraints.

Step E: validate

Walk through the requirements with students, coordinators, administration, and IT. Derive tests for event visibility, valid registration, full capacity, unauthorised publishing, and performance.

Step F: manage changes

Record each change request, assess impact, approve or defer it, update the requirement version, and update affected design and test artefacts.

> **Lesson:** the process is successful when the team has a shared, approved, testable understanding of the problem and can control changes—not merely when a document has been written.

12. Common Mistakes and How to Avoid Them

Mistake	Why it causes trouble	Better practice
Assuming the user's solution is the real need	The proposed feature may not solve the underlying problem.	Ask about goals, current work, pain points, and alternatives.
Writing vague requirements	Developers and testers interpret them differently.	Use measurable, observable acceptance criteria.
Talking only to the sponsor	Important user groups and operational constraints are missed.	Identify and consult all relevant stakeholder groups.
Ignoring exceptions	Real workflows fail at edge cases.	Ask about capacity, duplicate submissions, cancellations, errors, and permissions.
Treating the SRS as unchangeable	New information is hidden or changes become uncontrolled.	Use versioning, impact analysis, and agreed change control.
Validating only at the end	Major misunderstandings are discovered too late.	Review and validate requirements iteratively.

13. Classroom Activity

Activity: 15-minute requirements discovery exercise

Work in pairs. One student acts as a student user; the other acts as a requirements analyst for CEMS.

• Minutes 1–3: write down the problem and identify at least three stakeholders.

• Minutes 4–8: conduct a short interview using at least five open-ended questions.

• Minutes 9–12: write three functional and two non-functional requirements.

• Minutes 13–15: improve one vague requirement so it becomes measurable and testable.

Expected output: stakeholder list, interview notes, five requirement statements, and one improved requirement. Be ready to explain where each requirement came from.

14. Knowledge Check

1. What is requirements engineering?

A disciplined process to discover, analyse, document, validate, and manage system requirements.

2. Name the main stages described in this chapter.

Inception/feasibility, elicitation, analysis/negotiation, specification, validation, and management.

3. Why are interviews and observation both useful?

Interviews reveal motivations and explanations; observation reveals actual behaviour and workarounds.

4. Improve this requirement: “The app must be fast.”

Example: “The event list shall load within two seconds for 95% of requests under the agreed normal-load test.”

5. What is the purpose of an SRS?

To provide a structured, shared description of the system’s requirements and constraints.

6. Explain the difference between validation and verification.

Verification checks whether a work product is specified correctly; validation checks whether it represents the right system for stakeholder needs.

7. Why use requirement IDs and traceability?

To track origin, implementation, testing, status, and change impact.

8. A coordinator requests cancellation after approval. What should happen first?

Record the change request and analyse its impact before deciding whether to approve, defer, or reject it.

15. Glossary

Term	Meaning
Requirement	A capability, behaviour, need, or constraint the system must satisfy.
Stakeholder	A person or group affected by or interested in the system.
Elicitation	Discovering requirements from stakeholders, documents, and real work.
Feasibility study	An assessment of whether a proposed solution is practical and worthwhile.
Analysis	Clarifying, classifying, prioritising, and checking requirements.
SRS	Software Requirements Specification; a structured requirements document.
Validation	Checking that requirements represent stakeholder needs.
Traceability	The ability to follow a requirement to its source and related artefacts.
RTM	Requirements Traceability Matrix.
Acceptance criteria	Conditions or evidence used to decide whether a requirement is satisfied.

16. Homework and Deliverables

Use the College Event Management System case study or another system approved by your instructor.

• Prepare a one-page problem statement, objectives, scope, assumptions, and out-of-scope items.

• Identify at least five stakeholders and describe each stakeholder’s main concern.

• Choose two elicitation techniques and write five questions you would ask.

• Write five functional and three non-functional requirements with unique IDs.

• Improve two vague requirements so they are measurable or otherwise verifiable.

• Create a small RTM linking at least three requirements to their source and a proposed test case.

• List two possible change requests and explain what parts of the project might be affected.

Suggested submission structure

1. Problem and scope
2. Stakeholder list
3. Elicitation plan and questions
4. Requirements catalogue
5. Validation checklist
6. Mini traceability matrix
7. Change-management examples

Next session: Functional and non-functional requirements, requirement classification, and writing testable requirement statements.